

Programming in R: Exercise 3

Felix Clouth, f.j.clouth@tilburguniversity.edu

General remarks & tips

- Write your code in an .R script
- Don't save your workspace; start "clean" by running `rm(list = ls())`
- Add comments to your code using #. For an example of what an R script should roughly look like, check the R scripts with solutions from previous weeks.
- **Read the document "Missing Values in R" on Canvas**

Part 1: Cleaning data

Download the data set `height.txt` from Canvas > Modules > Week 3. This file contains the responses of 141 psychology students (first column) to a questionnaire. The data set contains the scores on 26 items (second through 27th column) and self-reported scores on the respondents' height (column 28), weight (column 29), gender (1 = male, 2 = female; column 30), and grade for an introductory course in ANOVA (column 31). The items in the questionnaire are related to height (Table 1) and could be answered with either "yes" (score 1) or "no" (score 0); it was expected that there is a positive relation between the total score and height. Thirteen items are positively worded, meaning that a score 1 is indicative for height, and 13 items are negatively worded (items indicated by an asterisk in Table 1), meaning that a score 0 is indicative for height. Each row contains the responses of one respondent. The value 9 indicates a missing score on an item or gender, the value 99 indicates a missing score on weight or grade, and the value 999 indicates a missing score on height.

Table 1

Items in the questionnaire. An asterisk indicates that the item was negatively worded.

No.	Question
1	I bump my head rather often.
2	The saddle on my bicycle is very low*.
3	My shoes would be too large for most people
4	In bed my cold feet bother me
5	My shoe size is one of the larger children's sizes*.
6	Literally, I look up to most of my friend*.
7	I think I would do pretty well in a basketball team.
8	When I take the bus, I can hardly reach the handles on the ceiling*.
9	When I am climbing stairs, I usually take two or more stairs at the time.

- 10 Other people will not easily strike an aggressive pose to me
 - 11 I would not make a very impressive police officer*.
 - 12 The back seat of a car is often too tight for me.
 - 13 I am physically uncomfortable in a school desk.
 - 14 Other people often literally look up to me.
 - 15 When I am walking, I often take more steps than most other people*.
 - 16 I often have to stand on my toes to look in the mirror*.
 - 17 When sitting at the table, I usually have not enough room for my legs
 - 18 When I am standing in line for a box office, I often see the shoulders of the person in front of me*.
 - 19 At parades, I am often in the front*.
 - 20 At the dinner table, I often bump my legs to other people's legs.
 - 21 People often literally look over me*.
 - 22 When I am waiting for a traffic light, I cannot remain seated on my bicycle*.
 - 23 In the train, I have to watch out not to bump my head on the doorpost.
 - 24 Others cannot easily find me on class photos (e.g., from high school)*.
 - 25 The servings in a restaurant of often not enough for me.
 - 26 In libraries, I have to use a stepladder to reach the books*.
-

1. Load the data `height.txt` in R and save it in an object. Make sure you check whether the data was loaded correctly (see the functions we discussed in Week 2).
2. To change a column name in a data frame, I can use the following general logic:

```
# Change name of 1st column of a data frame called df to "a"  
colnames(df)[1] <- "a"
```

Use the function `paste()` and the logic above to change the variable names of the 26 items (so not all items in the data frame!) to Q1, Q2, ..., Q26. Note that there should be no space between the Q and the number (check the help page for info).

3. Recode gender into 'm', 'f', or '?'. (Note: check the intro text to see how any missing values were coded for this variable.)
4. Check whether there are impossible values in gender (how can you do this quickly in R?). If so, recode these into '?' as well.

5. Recode the missing values (i.e., 9, 99, and 999) into NA (leave '?' for gender as is). Make sure you only do this for the relevant columns (i.e., the ID number of the 9th participant should not be missing)!
 6. Replace the missing values of grade and height by the mean of the variable.
 7. Calculate whether the total score of the questionnaire indeed correlates with height. Is this correlation statistically significant at $\alpha = .05$? Note that you have to reverse code the negatively worded items. As an extra (rather useless) challenge, try to code this in a single line (without the use of ";")!
-

Part 2: Summarizing data

8. Download the file big5.RData from Canvas > Modules > Week 3, and load it into R. (Check the slides/exercises/solutions of Week 2 if you don't remember how to load a .RData file.)
9. There should now be a data frame object big5 in your workspace. When something is "in your workspace", this means that you can refer to it by name.
 - a. How many variables and observations does the data frame have?
 - b. The participants filled out their answers on a Likert scale. How many categories did the questions have? I.e., was it a 5-point Likert scale, a 7-point Likert scale, ... etc?
10. The data set is a measurement of the Dutch translation of the NEO-PI-R on 500 first year psychology students (Dolan, Oort, Stoel, Wicherts, 2009). As you probably already know, the Big 5 personality factors are OCEAN: openness, conscientiousness, extraversion, agreeableness, and neuroticism (sometimes neuroticism is called "emotional stability").
 - a. Make R show the names of the columns.
 - b. Make R show only the means of every column. *Hint: note that the last column contains the factor gender.*
11. Using indexing (*hint: use seq() or :*), calculate the row (student's) means for the agreeableness indicators only.
 - a. Assign the result to a vector called student_agr.
 - b. We are interested in the degree of variation in indicators that measure agreeableness for each person. To get more insight, calculate each student's standard deviation for the agreeableness indicators, and assign it to a variable student_sds.
12. There is also a variable gender in the data frame (0 is male and 1 is female).
 - a. Using tapply(), show the mean of student_agr for the two genders. What is the difference in means between male and female students?
 - b. Using tapply(), show the standard deviation of student_agr for the two genders. What is the difference in standard deviation between male and female students?

- c. Using `tapply()`, show the mean of `student_sds` for the two genders (just do the same as in 12a but with `student_sds`). What do these numbers show? What is the difference with 12b?
13. I am interested in the answer patterns of the students on the questions for openness. Specifically, I'm interested to know how many questions the students answered with the maximum score of 5. We have already seen that you can use `apply()` to calculate something for each student, e.g., `apply(data, 1, mean)`. In this case, we could use the existing function `mean()` and implement it in `apply()`. However, there is no standard function in R to detect how often each student selected a 5. For this, I need to make my own custom function.
- a. Finish the code below by replacing the dots with the correct code to make a custom function to calculate the percentage of 5s in a vector. *Hint: the **name** of the function below is `perc_5`, and the **input** is `scores`, which should be a numeric vector.*
- ```
perc_5 <- function(scores){
 sum(... == 5)/...()
}
```
- b. Create a new data frame called `open` with only the answers to the Openness questions.
  - c. Use `apply()` and your custom function to calculate for each student (each row) what percentage of the openness question they answered with 5.
  - d. Which student had the highest percentage of 5s in their answers?
14. We have learned using conditional selection using operators like `>`, `==`, and `is.na()` to sub-select pieces of data that meet a particular condition. However, these operator may not always be the easiest to use when sub-selecting in certain scenarios.

One common scenario where the conditional selection operators we already looked at may not be efficient is in the case of strings. Often when sub-selecting strings, we want to know all elements that contain a certain word or characters like "Agreeableness" to select all agreeableness related questions.

- a. Run the following code to load in a version of the built-in `mtcars` data set that is (annoyingly) shuffled.

```
shuffling the mtcars dataset
set.seed(120225)
mtcars_shuffled <- mtcars[sample(nrow(mtcars)),]
```

Select all of the Mercedes (Merc) cars in the shuffled dataset. Use the `grep1()` function to sub-select the dataset.

Another common scenario where the conditional selection we already looked at may not be efficient is when you want to know which elements match to one value in a set of pre-specified values. You could use the `==` operator for each of those values, and then combine the results. However, this will take a while and look messy.

- b. Instead, use the `%in%` operator or `match()` function to find all cars in the shuffled dataset that have a rear axle ratio equal to one of the values in the vector below.

```
desired_axle_ratio <- c(3.07, 3.21, 3.15, 3.62, 3.85, 3.92, 4.08, 4.43)
```

Note, because `%in%` is an operator you can use `? "%in%"` to find help understanding what the operator does.

---